PYTHON FOR PHYSICS / PRACTICE PROJECT

Position and linear drag

One notebook | No plot required | Solutions withheld

Begin by reconstructing the Euler updates from memory, then consult your previous code as needed. Prioritize Part 1 if you get stuck.

1 / TRACK POSITION UNDER CONSTANT GRAVITY

Take downward as positive. Use g = 9.81 m/s^2 and y(0) = v(0) = 0.

dy/dt = v **dv/dt = g**

Extend your velocity function to return NumPy arrays of times, positions, and velocities, including the initial point. Calculate both Euler changes using the state at the start of the step.

Run to 2 s with dt = 0.10 s (20 steps) and dt = 0.05 s (40 steps).

For each run, print the final position and its absolute error against:

y_exact(t) = (1/2) g t^2

Explain whether position is overestimated or underestimated and how halving dt changes the error. Predict the direction before running.

2 / ADD LINEAR DRAG

Use mass m = 0.20 kg and drag coefficient b = 0.10 kg/s.

dy/dt = v **dv/dt = g - (b/m) v**

Modify your function to accept dt, n_steps, m, and b. Keep the same initial conditions. Run to 10 s with dt = 0.20 s and 50 steps.

3 / CHECK THE VELOCITY

tau = m/b **v_terminal = mg/b**

v_exact(t) = v_terminal [1 - exp(-t/tau)]

Print the maximum absolute velocity error at matching times and the final simulated velocity. Write one sentence assessing physical plausibility.

CHECKS AND SUBMISSION

Keep time local; use integer step counts; store matching times and states. With drag, velocity should rise toward terminal velocity. tau is in seconds; terminal velocity is in m/s. Send your notebook and results, even if unfinished.

Focus: state alignment, units, update order, and quantitative error checks.